P981 - ARCH - Backend Processing Architecture (Draft)

- 요구사항
- 요약
- 실행 흐름
- 스테이지
  - Stage 1 — DISCOVER
  - Stage 2 — SYNTHESIZE
- 모듈
  - 도메인 영역 (Domain)
  - 운영 영역 (Runtime / Operations)
- 그 외 기능 고려 사항
  - 인프라 영역 (Infrastructure)
  - 에러 처리 및 복구
  - 보안 및 권한
  - 확장성 고려사항
- 규칙 몇가지 정의
  - 자산 보존 규칙 (중요)
  - Moment 선택 원칙
- 외부 인터페이스
  - Stage 1 — DISCOVER
  - Stage 2 — SYNTHESIZE
- API Example
  - POST /api/v1/synthesize
  - GET /api/v1/synthesize/{job_id}
  - POST /api/v1/discover
  - GET /api/v1/discover/{job_id}
- 내부 인터페이스
  - Stage 1 — DISCOVER (내부)
  - Stage 2 — SYNTHESIZE (내부, 스타일별)
- 공통 규칙
- 그 외 고려 사항

요구사항

P981 시스템은 파크의 레이스981을 탑승하는 동안 촬영된 고객의 **정면 비디오**로부터
의미 있는 **Moment(순간)** 를 자동으로 발견하고 고객 히스토리에 **누적**한 뒤,
요청 시 해당 Moment를 기반으로 **시네마틱 라이브 포트레이트**를 생성한다.

- 고객은 여러 번 탑승 가능 → 고객 히스토리로 누적
- 원본 비디오는 7일 후 삭제 → Stage 1에서 재사용 자산을 반드시 추출/보존

요약

- Stage는 2개: **DISCOVER(자동/비동기)** / **SYNTHESIZE(요청/스타일별)**
- 도메인 모듈은 6개: **media/state/moment/asset/synthesis/stage**

- 외부 인터페이스: `/api/v1/discover`, `/api/v1/synthesize` + job status 조회
- 내부 인터페이스(모듈 실행 순서): `media→state→moment→media→asset`, `asset→synthesis→asset`

실행 흐름

```
비디오 업로드
   ↓
Stage 1 : DISCOVER   (자동 / 비동기)
:media → state → moment → media → asset
   ↓
Customer Moment History + 재사용 자산
   ↓
Stage 2 : SYNTHESIZE (요청 기반 / 스타일별)
:asset → synthesis → asset
   ↓
이미지 / 시네마틱 영상
```

스테이지

Stage 1 — DISCOVER

트리거: 비디오가 스토리지(S3 등)에 업로드됨
목표: 고객 비디오에서 의미 있는 Moment를 찾아 고객 히스토리에 누적

처리 흐름:

1. 고객 상태 타임라인 생성 (감정 상태, 손 위치, 얼굴 품질 등)
2. 다수의 Moment 후보 생성
3. 중복 제거 + 다양성 보장 조건을 만족하는 N개 Moment 선택
4. 선택된 각 Moment에 대해:
   - 대표 프레임 묶음(Keyframe Pack) 추출
   - 해당 순간의 짧은 비디오 클립(Moment Clip) 추출
5. 모든 결과를 저장하고 고객 단위 히스토리에 누적

캐싱: 동일 비디오 재처리 방지를 위해 미디어 해시 기반 캐시 확인

👉 **이 단계가 끝나면 원본 비디오 없이도 이후 모든 작업 수행 가능**

에러 처리:

- 추출 실패 시 최대 3회 재시도
- 재시도 실패 시 DLQ로 이동 및 알림 발송

Stage 2 — SYNTHESIZE

트리거: 고객 또는 시스템 요청

목표: 저장된 Moment를 입력으로 요청된 스타일의 결과물 생성

실행 방식:

- 하나의 고정된 파이프라인이 아님
- 요청된 스타일 단위로 독립적으로 실행됨
- 이미 생성된 결과물은 재사용됨 (중복 생성 방지)

지원되는 생성 유형:

- **Base Portrait 생성** (리터칭 및 정준화)
- **Close-up 이미지 생성**
- **Full-body 이미지 생성**
- **Cinematic Live Portrait 생성** (Close + Full을 활용한 줌인 연출 영상)

최적화:

- Base Portrait는 필요할 때만 생성
- 생성 완료 시 메타데이터 DB에 인덱싱하여 빠른 검색 지원

알림:

- 생성 완료 시 요청자에게 웹훅/이메일 알림
- 실패 시 에러 상세 정보와 함께 알림

모듈

도메인 영역 (Domain)

Module 1 — p981-media

역할: 미디어 등록 및 프레임/클립 추출

책임:

- 원본 비디오 접근 및 미디어 단위 추출만 수행
- 판단, 선택, 분석 로직 없음

캐시 활용: 동일 미디어 추출 요청 시 캐시된 결과 반환

Module 2 — p981-state

역할: 고객 상태 타임라인 생성

책임:

- emotion / hand / quality 등 시간 기반 상태 관측
- moment 판단 또는 선택 로직 없음

출력: 시간별 상태 데이터 (JSON 타임라인)

Module 3 — p981-moment

역할: Moment 분석 및 결정

책임:

- Moment 후보 생성
- 중복 제거 및 다양성 조건을 만족하는 N개 Moment 선택
- 미디어 추출 직접 수행하지 않음

알고리즘:

- 클러스터링 기반 중복 제거
- 품질 점수 + 다양성 점수 조합으로 최종 선택

Module 4 — p981-asset

역할: 자산 및 히스토리 관리

책임:

- 모든 중간/최종 산출물 저장 및 조회
- 고객 Moment 히스토리 누적 및 인덱싱

저장소:

- S3: 실제 미디어 파일
- Metadata DB: 검색 가능한 메타데이터 (태그, 타임스탬프, 품질 점수 등)

인덱싱: 고객 ID, 날짜, 스타일, 품질 점수 등으로 빠른 검색 지원

Module 5 — p981-synthesis

역할: 결과물 생성

책임:

- 요청된 스타일(Base / Close-up / Full-body / Cinematic)에 따른 생성 수행
- Moment 선택 또는 분석 로직 없음

생성 전략:

- 스타일별 독립적인 파이프라인
- 의존성 있는 스타일(Cinematic → Close-up + Full-body)은 자동으로 선행 작업 확인

Module 6 — p981-stage

역할: 도메인 스테이지 정의

책임:

- Stage가 어떤 도메인 모듈들의 조합으로 구성되는지 정의
- 입력/출력 계약 및 실행 순서 명시

- 실행 방식(API, Job, Worker)에는 관여하지 않음

정의:

- `DiscoverStageDefinition`
- `SynthesizeStageDefinition`

운영 영역 (Runtime / Operations)

Module 7 — p981-runtime

역할: Stage 실행 및 운영

책임:

- REST API 제공 (Stage 실행 요청 / 상태 조회)
- Job 생성 및 상태 관리
- Stage Runner 실행 (p981-stage 정의 기반)
- Worker / Scheduler / Watcher 관리

API 엔드포인트:

```
POST   /api/v1/discover          # Discover Stage 실행 요청
POST   /api/v1/synthesize        # Synthesize Stage 실행 요청
GET    /api/v1/jobs/{jobId}      # Job 상태 조회
DELETE /api/v1/jobs/{jobId}      # Job 취소
GET    /api/v1/jobs              # Job 목록 조회 (필터링 지원)
```

Job 상태:

- `PENDING` : 대기 중
- `RUNNING` : 실행 중
- `COMPLETED` : 완료
- `FAILED` : 실패
- `RETRYING` : 재시도 중

Scheduler:

- 우선순위 기반 작업 할당
- 동시 실행 수 제어 (고객별, 전체)
- 리소스 사용량 모니터링

Watcher:

- S3 이벤트(ObjectCreated) 자동 감지
- 이벤트 → Job 자동 생성

그 외 기능 고려 사항

(아래는 AI 에서 긁어온 내용. 검토후 정리 필요함)

인프라 영역 (Infrastructure)

Cache Layer

역할: 중복 처리 방지 및 성능 최적화

캐시 대상:

- 미디어 추출 결과
- 상태 분석 결과
- 생성된 Base Portrait

전략: 미디어 해시 기반 캐시 키, TTL 7일

Metadata DB

역할: 검색 최적화 및 히스토리 관리

스키마 예시:

```
- customer_id
- moment_id
- timestamp
- quality_score
- tags (emotion, quality_level, etc.)
- asset_refs (S3 paths)
- synthesis_history
```

인덱스: customer_id, timestamp, quality_score, tags

Notification Service

역할: 작업 완료/실패 알림

알림 채널:

- 웹훅 (HTTP POST)
- 이메일
- 슬랙 (내부 운영팀)

알림 조건:

- Job 완료
- Job 실패 (재시도 소진)
- 긴급 에러 (시스템 장애)

Monitoring & Logging

역할: 시스템 관찰성 및 문제 진단

수집 메트릭:

- Job 실행 시간 (스테이지별, 모듈별)
- 성공/실패율
- 재시도 빈도
- 리소스 사용량 (CPU, 메모리, 스토리지)

- API 응답 시간

로그 레벨:

- INFO: 정상 흐름
- WARN: 재시도 가능한 에러
- ERROR: 치명적 에러
- DEBUG: 상세 진단 정보

대시보드: Grafana + Prometheus 기반 실시간 모니터링

에러 처리 및 복구

Retry Handler

- 일시적 실패(네트워크, 리소스 부족) 발생 시 자동 재시도
- 지수 백오프(Exponential Backoff) 전략
- 최대 3회 재시도

Dead Letter Queue (DLQ)

- 재시도 소진 후에도 실패한 Job 저장
- 수동 재처리 또는 분석 가능
- 주기적으로 DLQ 모니터링 및 알림

보안 및 권한

API 인증

- API Key 기반 인증
- 고객별 접근 제어 (자신의 데이터만 조회 가능)

데이터 보호

- S3 서버사이드 암호화 (SSE-S3)
- 전송 중 암호화 (HTTPS/TLS)
- 메타데이터 DB 암호화

확장성 고려사항

수평 확장

- Worker Pool 동적 스케일링 (부하 기반)
- API 서버 로드 밸런싱

성능 최적화

- 캐시 레이어로 중복 처리 최소화
- 메타데이터 DB 인덱싱으로 검색 속도 개선
- 비동기 처리로 API 응답 시간 단축

비용 최적화

- S3 Lifecycle Policy로 오래된 중간 산출물 자동 삭제

- 캐시 TTL 조정으로 스토리지 비용 절감
-

규칙 몇가지 정의

- Stage 1 (DISCOVER) 종료 시, 원본 비디오 없이 Stage 2를 수행할 수 있어야 한다.
- 이를 위해 각 Moment는 반드시 다음 자산을 포함한다:
  - Keyframe Pack (합성 입력 프레임)
  - Moment Clip (비디오 맥락)
  - Moment Metadata (시간 구간, 타입, 중복 판별 정보)
- 모든 Stage 간 데이터 전달은 asset_ref 기반으로 수행된다.
- 원본 비디오는 Stage 1에서만 참조 가능하며, 이후 단계에서는 사용하지 않는다.

자산 보존 규칙 (중요)

원본 비디오가 7일 후 삭제되므로
Stage 1 종료 시 아래 자산은 **반드시 보존**되어야 한다.

- 대표 프레임 묶음 (Keyframe Pack)
- Moment 비디오 클립 (Moment Clip)
- Moment 메타데이터
  (시간 구간, 타입, 점수, 중복 판별용 정보 등)

이 자산들이 있으면 이후 모든 생성 작업을 재수행할 수 있다.

Moment 선택 원칙

- 하나의 비디오에서 여러 Moment가 발견될 수 있음
- 최종 저장되는 Moment는 항상
  - 중복되지 않으며
  - 시간적·의미적으로 다양성을 만족
- 중복 판별은
  - 동일 비디오 내
  - 동일 고객의 과거 히스토리 전체
    기준으로 수행됨

외부 인터페이스

Stage 1 — DISCOVER

```
[DISCOVER Job Request]
  - video_ref
  - customer_id
        ↓
[DISCOVER API Response]
  - job_id
  - status = accepted | running | completed
        ↓
[DISCOVER Job Result]   // (asset에 저장된 결과 참조)
  - moment_refs
  - keyframe_pack_refs
  - moment_clip_refs
  - history_updated = true
```

Stage 2 — SYNTHESIZE

(a) Base 생성

```
[SYNTHESIZE Job Request]
  - style = base
  - input = moment_ref   // 또는 customer_id + selection_policy
        ↓
[SYNTHESIZE API Response]
  - job_id
  - status = accepted | running | completed
        ↓
[SYNTHESIZE Job Result]
  - base_portrait_ref
  - reused_existing = true | false

```

(b) Close-up 생성

```
[SYNTHESIZE Job Request]
  - style = closeup
  - input = base_portrait_ref
        ↓
[SYNTHESIZE Job Result]
  - closeup_image_ref
  - reused_existing = true | false

```

(c) Full-body 생성

```
[SYNTHESIZE Job Request]
  - style = fullbody
  - input = base_portrait_ref
        ↓
[SYNTHESIZE Job Result]
  - fullbody_image_ref
  - reused_existing = true | false

```

(d) Cinematic 생성

```
[SYNTHESIZE Job Request]
  - style = cinematic
  - input = closeup_image_ref + fullbody_image_ref
        ↓
[SYNTHESIZE Job Result]
  - cinematic_video_ref
  - reused_existing = true | false

```

API Example

```
Client
  ↓ POST /discover
DISCOVER Job
  ↓ (side effects)
asset (Moment / KeyframePack / Clip)

Client
  ↓ POST /synthesize
SYNTHESIZE Job
  ↓ (side effects)
asset (Base / Image / Video)

```

POST `/api/v1/synthesize`

```
[Request]
{
  style: "base" | "closeup" | "fullbody" | "cinematic",

  // style별 입력
  moment_ref?: string,                 // base 생성 시
  base_portrait_ref?: string,          // closeup / fullbody
  closeup_image_ref?: string,          // cinematic
  fullbody_image_ref?: string          // cinematic
}

```

```
[Response]
{
  job_id?: string,
  status: "accepted" | "completed",
  generated_asset_ref?: string,
  reused_existing?: true | false
}

```

Synthesis 요청은 처리 시간에 따라 동기 또는 비동기로 수행될 수 있다.

GET `/api/v1/synthesize/{job_id}`

```
[Response]
{
  status: "running" | "completed" | "failed",
  result?: {
    generated_asset_ref: string,
    reused_existing: true | false
  }
}

```

POST `/api/v1/discover`

```
[Request]
{
```

```
  video_ref: string,
  customer_id: string
}

```

```
[Response]
{
  job_id: string,
  status: "accepted" | "running" | "completed"
}
```

`GET` `/api/v1/discover/{job_id}`

```
[Response]
{
  status: "running" | "completed" | "failed",
  result?: {
    moment_refs: string[],
    keyframe_pack_refs: string[],
    moment_clip_refs: string[],
    history_updated: true
  }
}
```

내부 인터페이스

Stage 1 — DISCOVER (내부)

```
[Stage Input]
  - video_ref
  - customer_id
        ↓
[media]
  IN  - video_ref
  OUT - video_access_ready   // 프레임/클립 추출 가능 상태

        ↓
[state]
  IN  - video_ref
  OUT - state_timeline_ref
        ↓
[moment]
  IN  - state_timeline_ref
  OUT - moment_candidate_refs
        ↓
[moment]
  IN  - moment_candidate_refs
      - customer_history_signatures (optional)
  OUT - selected_moment_refs (N, dedupe + diverse)
        ↓
[moment → media]
  IN  - selected_moment_refs
  OUT - extraction_plan
       - per_moment: time_range
       - per_moment: keyframe_timestamps
        ↓
[media]
  IN  - video_ref + extraction_plan
```

```
  OUT - keyframe_pack_refs
      - moment_clip_refs
        ↓
[asset]
  IN  - selected_moment_refs
      - keyframe_pack_refs
      - moment_clip_refs
  OUT - moment_refs
      - history_updated = true

```

Stage 2 — SYNTHESIZE (내부, 스타일별)

(a) Base 생성 내부

```
[Stage Input]
  - style = base
  - moment_ref
        ↓
[asset]
  IN  - moment_ref
  OUT - keyframe_pack_ref
        ↓
[synthesis]
  IN  - keyframe_pack_ref
  OUT - base_portrait_ref
        ↓
[asset]
  IN  - base_portrait_ref
  OUT - stored_ref + reused_existing

```

(b) Close-up / Full-body 생성 내부

```
[Stage Input]
  - style = closeup | fullbody
  - base_portrait_ref
        ↓
[synthesis]
  IN  - base_portrait_ref
  OUT - image_ref
        ↓
[asset]
  IN  - image_ref
  OUT - stored_ref + reused_existing

```

(c) Cinematic 생성 내부

```
[Stage Input]
  - style = cinematic
  - closeup_image_ref
  - fullbody_image_ref
        ↓
[synthesis]
  IN  - closeup_image_ref + fullbody_image_ref
  OUT - cinematic_video_ref
        ↓
[asset]
  IN  - cinematic_video_ref
  OUT - stored_ref + reused_existing

```

공통 규칙

```
- API Response는 "요청 수락/상태/참조"를 반환하고, 실제 산출물은 asset에 저장된다
```

```
- Stage 간 데이터 전달은 asset_ref 기반으로만 이루어지며, 원본 비디오는 DISCOVER

```

그 외 고려 사항

- Moment는 항상 두 가지를 포함
  - 프레임 입력 (KeyframePack)
  - 비디오 맥락 (MomentClip)
- Stage 1은 누적만 수행 (Base 생성 ❌)
- Base 및 시네마틱 생성은 전부 On-Demand
- 중복된 Moment는 자동 제거, 결과는 항상 다양성 확보
- 원본 비디오 삭제 이후에도 재생성 가능
- 모듈 간 전달은 blob(이미지/비디오) 자체가 아니라 asset_ref + metadata로
- 모든 시간 기준은 Video time (seconds, float) 로 통일
- 모든 “관측치 타임라인”은 (t, value, confidence) 형태로 통일
- “결정(policy)”은 moment-analysis 내부로 고정 (다른 모듈로 새지 않게)